# Modern Computer Vision Architecture

Backbones, Heads and Pretraining

Amit Kumar Saha (A00336661)
11/04/2026

# Contents

# Figures

# 1. Introduction

In modern computer vision, a model is often split into two parts: a **backbone** and a **head**.

The **backbone** is the main feature extractor. It takes an input image and converts it into increasingly useful internal representations. Early layers usually capture local patterns such as edges, corners, and textures, while deeper layers capture higher-level structures such as object parts, shapes, and semantic content. Because this transformation is broadly useful across many tasks, the backbone is often pretrained once and then reused.

The **head** is the task-specific part attached to the backbone. It takes the backbone features and turns them into outputs for a particular problem. For image classification, the head may be a linear layer that predicts class probabilities. For object detection, the head may predict bounding boxes and categories. For segmentation, the head may output a mask for each pixel. The key idea is that the backbone provides general-purpose visual features, while the head adapts those features to the final task.

This design is attractive because it separates general visual understanding from task-specific prediction. Instead of training every model from scratch, we can keep the same backbone and swap or fine-tune different heads depending on whether we want classification, detection, segmentation or depth estimation.

Two common examples to show how the same backbone can support multiple tasks.

## Example 1: ResNet as a shared backbone

ResNet was originally introduced for image classification on ***ImageNet***, where a classification head maps global image features to class labels [1]. Later, the same ResNet backbone was reused in detection and segmentation systems such as ***Faster R-CNN*** and ***Mask R-CNN*** [2, 3]. In those systems, the head changes: Faster R-CNN adds region proposal and detection heads, while Mask R-CNN adds an extra mask prediction head for instance segmentation. The backbone still extracts strong features, but the head determines the task.

## Example 2: Vision Transformer backbones reused across tasks

Vision Transformer (ViT) style backbones are also reused broadly [4]. A ViT backbone can be used with:

- A classification head for image recognition,
- A detection head in transformer detectors such as **DETR**-like systems,
- A segmentation decoder or mask head for semantic or instance segmentation,
- A depth prediction head for monocular depth estimation.

The same pattern also appears in self-supervised models such as DINO, where the pretrained backbone is later fine-tuned or probed with different downstream heads [5].

The backbone + head idea is illustrated with a simple example of an image classification model in Figure 1:

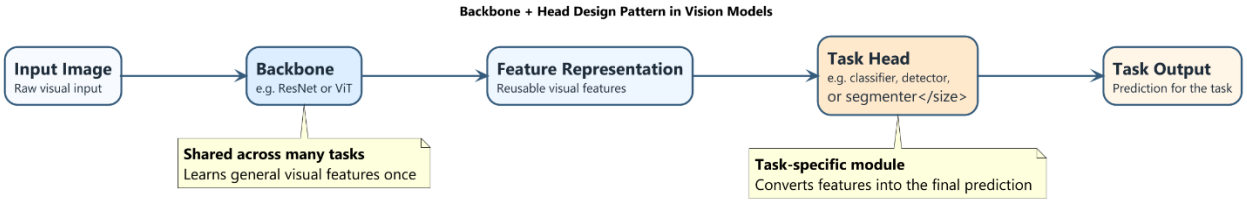


*Figure 1 Backbone + Head Design Pattern in Vision Models*

***Figure 1*** highlights the modular structure: the backbone extracts reusable features, and the head converts them into the output needed for a specific task.

# 2. Supervised and Self-supervised pretraining

Pretraining refers to training a model on a large dataset before adapting it to a downstream task. The main difference between **supervised pretraining** and **self-supervised pretraining (SSL)** is the source of the training signal. Both approaches can produce strong backbones, but they encourage somewhat different representations.

## 2.1 Supervised pretraining

In supervised pretraining, the model learns from human-provided labels such as object categories or bounding boxes.

### Strategy 1: ImageNet classification pretraining

A standard supervised strategy is to train a backbone for image classification on ***ImageNet***, as done for models such as ***VGG*** and ***ResNet*** [1, 6]. The model takes an image and predicts one class label from a fixed taxonomy.

The **training signal** is the ground-truth class label for each image. This is explicit human annotation.

The **features encouraged** by this setup are mainly category-level semantic features. The model is rewarded for being invariant to nuisance factors such as small viewpoint changes, lighting changes, or background clutter, as long as it can still predict the correct class. This often produces strong general-purpose features, which is why ImageNet-pretrained backbones became a standard starting point for many downstream tasks.

### Strategy 2: Supervised detection pretraining on COCO

Another supervised strategy is to train a backbone jointly with a detection head on an object detection dataset such as ***COCO***, for example in ***Faster R-CNN*** or ***Mask R-CNN*** [2, 3].

The **training signal** includes human-annotated bounding boxes, object classes, and possibly segmentation masks.

The **features encouraged** in this case are more object-level and spatially localized than in pure classification. The model must not only recognize object categories but also preserve enough spatial detail to localize them. As a result, detection-style supervised pretraining may produce features that are especially useful for localization-heavy downstream tasks.

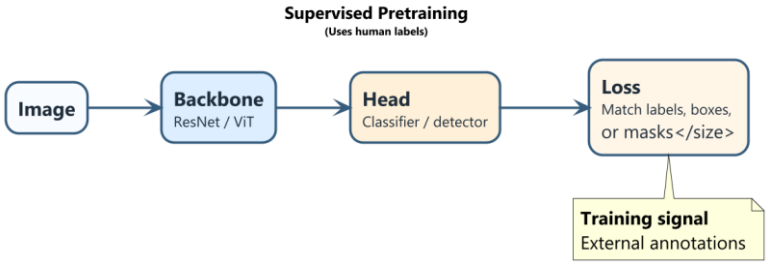


*Figure 2 Supervised pretraining flow*

## 2.2 Self-supervised pretraining

In self-supervised pretraining, the model learns from structure already present in the data, without manual labels.

### Strategy 1: Siamese / contrastive learning

A widely used self-supervised pretraining strategy is ***Siamese*** or ***contrastive learning***, including ***SimCLR*** and ***MoCo*** [7, 8]. The core idea is to create two different augmented views of the same image, for example by random cropping, colour jittering, or flipping. These two views should produce similar representations, because they come from the same underlying image. At the same time, representations from different images are encouraged to be distinct.

The **training signal** is therefore not a human label but the relationship between the samples: two views of the same image are treated as positives, while views from different images are treated as negatives or contrasted indirectly.

The **features encouraged** are invariance to the chosen augmentations and the ability to group semantically similar content while separating unrelated images. If the augmentations are designed well, the backbone learns robust representations without ever seeing category labels.

### Strategy 2: DINO-style self-distillation

**DINO** is another influential self-supervised pretraining strategy based on self-distillation [5]. It uses a student network and a teacher network. Both receive different views of the same image. The teacher produces a target distribution over features, and the student learns to match it. The teacher itself is updated from the student using an exponential moving average, so no human labels are needed.

The **training signal** is the consistency between teacher and student predictions across different views of the same image.

The **features encouraged** include invariance across augmentations, grouping of semantically related images, and surprisingly strong object-level structure. DINO-style methods are especially notable because attention maps in transformer backbones often highlight coherent objects, even though no object labels or masks were provided during training.

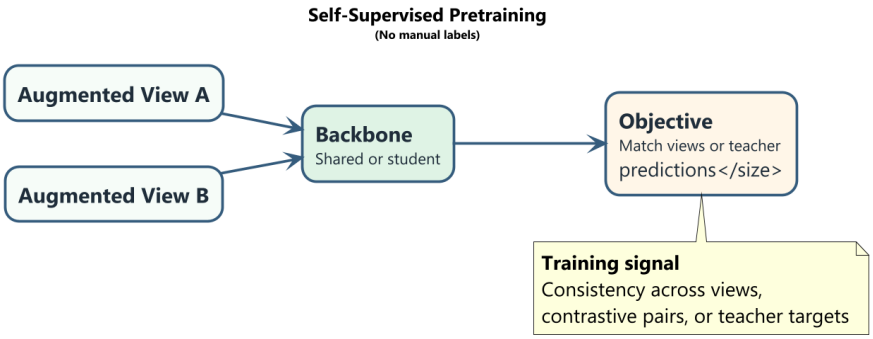


*Figure 3 Self-supervised pretraining flow*

## 2.3 Comparison - Supervised and Self-Supervised Pretraining

One key difference is the **source of supervision**. Supervised pretraining relies on external annotations such as class labels or boxes. Self-Supervised pretraining derives supervision from the data itself, for example by matching two views of the same image or matching student and teacher outputs.

Another key difference is the **type of representation learned**. Supervised pretraining often learns features aligned with the label space used during training. For example, ***ImageNet*** supervision strongly encourages category discrimination among its classes. Self-Supervised pretraining is less tied to a fixed human-defined label space and may learn more general-purpose representations that capture visual similarity, invariance, or latent object structure.

An **advantage** of Self-Supervised pretrained backbones is that they can exploit very large unlabelled datasets, which are much easier and cheaper to collect than dense manual annotations. This makes SSL attractive when labelled data is scarce or expensive.

A **limitation** is that Self-Supervised pretraining objectives do not always align perfectly with the downstream task. The learned invariances may sometimes discard details needed later, and additional fine-tuning or task-specific adaptation may be required. In practice, Self-Supervised pretrained backbones can be very strong, but their success depends heavily on the pretraining objective, augmentations, and downstream setup.

# 3. Case study: SAM as backbone + head

The ***Segment Anything Model (SAM)*** is a useful case study because it clearly follows the backbone + head design while adding prompts as an important interface [9].

At a high level, the **image encoder** in SAM acts as the backbone. It processes the input image once and produces a rich image representation. This representation is intended to be broadly reusable, rather than tied to only one fixed segmentation output.

The **mask decoder** acts as the task-specific head. It combines image features with prompt information and predicts segmentation masks. The prompt encoder is also an important part of the system because it converts user or task prompts, such as points, boxes, or input masks, into a form the mask decoder can use.

In this design, **prompts** play the role of task conditioning. A point prompt may indicate "segment the object near here." A box prompt may indicate "segment the object inside this region." An input mask prompt may refine an existing segmentation. The backbone does not change, but the prompt and decoder interaction changes what segmentation result is produced.

SAM illustrates the backbone + head idea very well:

- The ***image backbone*** is trained to produce strong general image features,
- The ***decoder head*** turns those features into segmentation outputs,
- ***Prompts*** let the same backbone support many segmentation-like behaviours without retraining the full system for each exact use case.

Conceptually, this is powerful because the expensive visual understanding is concentrated in the backbone, while the head remains flexible. Rather than building a separate segmentation model for every variant of the task, SAM reuses one strong image representation and changes the conditions under which the head predicts masks.

***Figure 4*** shows SAM using backbone, head and prompt for image processing.

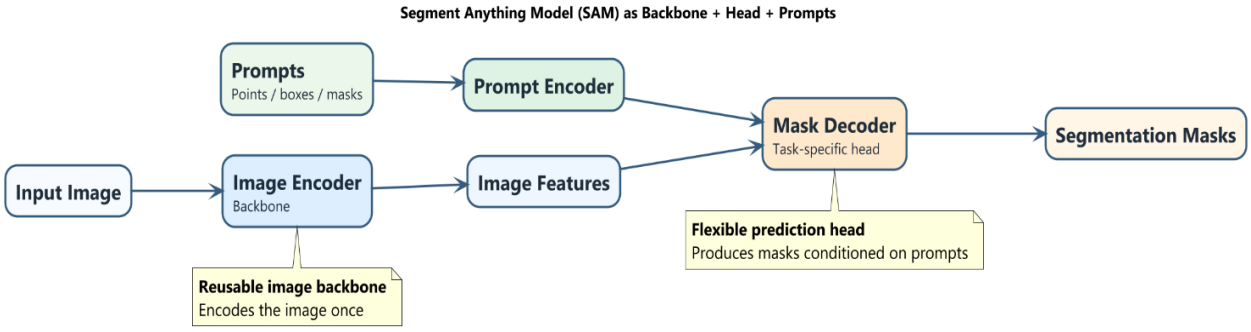


*Figure 4 SAM as Backbone + Head + Prompts*

# 4. Limitations - Generic backbones

A good example of a task where a generic ImageNet-style backbone is not clearly ideal is **image super-resolution**.

Super-resolution aims to reconstruct a high-resolution image from a low-resolution input. This requires recovering very fine spatial detail, including textures, edges, and precise pixel relationships. A frozen generic backbone such as ResNet, DINO, or SAM may not be sufficient here because those models are usually trained to become **invariant** to many image changes. That invariance is helpful for recognition tasks, but super-resolution needs sensitivity to tiny local differences.

In other words, high-level semantic features are not enough. A model may correctly recognize that an image contains a face, a car, or a dog, but that does not tell it exactly which high-frequency details should be reconstructed at the pixel level. The task depends strongly on local image statistics and on the mapping between degraded and clean images.

For super-resolution, the model often needs different inductive biases:

- preservation of exact spatial correspondences,
- sensitivity to high-frequency texture and edge structure,
- an understanding of the degradation process that created the low-resolution image,
- architectures designed for dense pixel reconstruction rather than semantic invariance.

This is why super-resolution systems often use specialized encoder-decoder or residual dense architectures trained directly for restoration, instead of simply attaching a small head to a frozen recognition backbone. The task is fundamentally about reconstructing missing pixel detail, not just extracting semantic meaning.

More broadly, similar concerns apply to denoising, deblurring, raw sensor processing, and optical flow. These tasks depend heavily on low-level image structure, motion consistency, or imaging physics. A generic backbone trained for semantic understanding may provide some useful context, but it is usually not the complete solution.

***Figure 5*** compares high-level ***recognition tasks*** with low-level ***reconstruction tasks***. Recognition models benefit from semantic, invariant features, while reconstruction tasks such as super-resolution need precise pixel-level detail and spatial accuracy. It highlights why generic backbones are often less suitable for restoration problems.

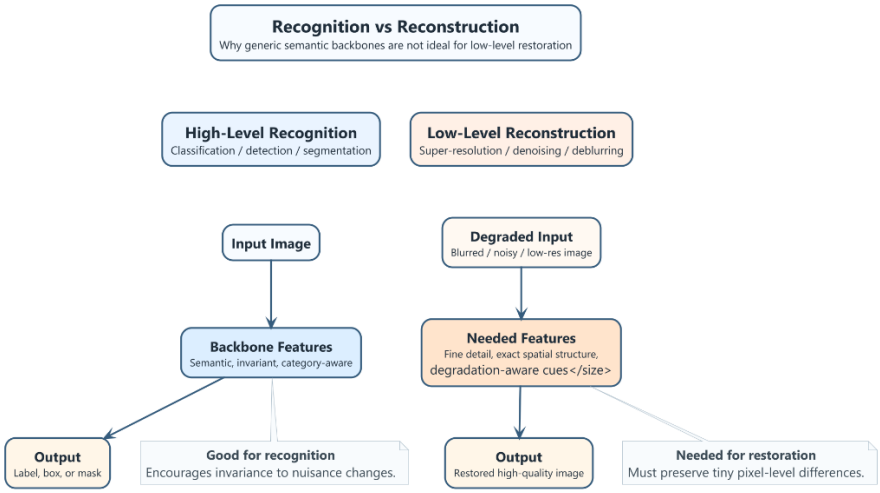


*Figure 5 Tasks - Recognition vs Reconstruction*

# Conclusion

The ***backbone + head*** pattern is one of the central design ideas in modern computer vision. A backbone learns reusable visual features, while a head maps those features to a specific task such as classification, detection, segmentation, or depth prediction. This modularity makes transfer learning practical and efficient.

Both supervised and self-supervised pretraining fit naturally into this pattern. Supervised pretraining uses human annotations and often learns label-aligned semantic features, while self-supervised pretraining uses structure within the data itself and can produce more general-purpose representations without manual labels.

Segment Anything Model (SAM) is a strong example of the backbone + head pattern in practice: a powerful image backbone is reused with a decoder head and prompts to support flexible segmentation behaviour. However, the pattern is not equally suitable for every problem. For low-level reconstruction tasks such as super-resolution, generic invariant backbones are often less appropriate because the task requires precise pixel-level detail and task-specific inductive biases.

# References

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," 2016.

[2] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," 2015.

[3] K. He, G. Gkioxari, P. Dollar, and R. Girshick, "Mask R-CNN," 2017.

[4] A. Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," 2021.

[5] M. Caron et al., "Emerging Properties in Self-Supervised Vision Transformers," 2021.

[6] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," 2015.

[7] T. Chen et al., "A Simple Framework for Contrastive Learning of Visual Representations," 2020.

[8] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum Contrast for Unsupervised Visual Representation Learning," 2020.

[9] A. Kirillov et al., "Segment Anything," 2023.